Manual del Programador - Piedra, Papel o Tijera v1.0.0

EEST N°1 de Vicente López - Laboratorio de Programación (LPR) - 5° 3° A-B - 2026

Autor: Lucas Del Pino

1. Descripción técnica

Programa de consola en C++ (estándar C++11) que implementa Piedra, Papel o Tijera contra la computadora y persiste cada partida en un archivo de texto con la librería estándar fstream. El archivo fuente único es src/main.cpp.

2. Entorno y compilación

- Sistema: Windows 10/11, VS Code, PowerShell.
- Compilador: g++ con el estándar C++11.

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

3. Estructura del proyecto

- src/main.cpp: código fuente del juego.
- data/: carpeta para el archivo persistente historial_partidas.txt.
- docs/: informe APA v7 y manuales.
- capturas/: evidencias de ejecución en formato png.
- README.md, CHANGELOG.md, LICENSE, .gitignore y .gitattributes.

4. Librerías utilizadas

- iostream: entrada y salida por consola (cin, cout, cerr).
- fstream: flujos de archivo (ofstream para escribir, ifstream para leer).
- string: manejo de cadenas de texto.
- cstdlib y ctime: rand() y srand(time(0)) para la elección aleatoria de la IA.

5. Modelo de datos: struct Partida

```
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

Agrupa los cinco datos heterogéneos de una partida en una sola entidad. Las funciones reciben la partida completa en lugar de cinco variables sueltas.

6. Funciones

- main(): inicializa la semilla aleatoria, muestra el menú en un bucle do-while y despacha cada opción (1 jugar, 2 historial, 3 salir).
- jugarPartida(int& contadorID): crea una Partida, asigna el ID e incrementa el contador por referencia, pide el nombre (cin.ignore() y getline), lee la jugada, genera la jugada de la IA con (rand() % 3) + 1, evalúa el resultado y llama a guardarEnDisco().
- guardarEnDisco(const Partida& p): abre el archivo con ofstream en modo ios::app, verifica is_open(), escribe un registro por línea en formato CSV y cierra el archivo.
- leerHistorialDisco(): abre el archivo con ifstream, lo recorre con getline y el delimitador coma, imprime cada registro y cuenta el total.

7. Reglas del juego

La partida es Empate si ambas elecciones son iguales. Es Victoria si Piedra vence a Tijera, Papel vence a Piedra o Tijera vence a Papel. En cualquier otro caso es Derrota.

8. Persistencia

Formato de cada línea del archivo (CSV):

```
ID,Jugador,EleccionJugador,EleccionIA,Resultado
1,Lucas,Piedra,Papel,Derrota
```

- ios::app agrega al final sin borrar el contenido anterior. Sin esa bandera, ofstream trunca el archivo en cada apertura y solo quedaría la última partida.
- El código abre la ruta src/historial_partidas.txt, por lo que debe ejecutarse desde la raíz del repositorio.
- Limitación: el nombre del jugador no debe contener comas, porque rompería el formato CSV al leer.

9. Manejo del búfer

Después de cin >> opcion queda un salto de línea (\n) en el búfer. Si se llamara a getline() directamente, leería ese salto como una línea vacía. Por eso se invoca cin.ignore() antes de getline(cin, p.jugador).

10. Limitaciones y mejoras posibles

- contadorID arranca en 1 en cada ejecución, por lo que los IDs se repiten entre sesiones. Mejora: leer el último ID del archivo al iniciar.
- No hay validación de la jugada: cualquier valor distinto de 1 o 2 se toma como Tijera.
- Si cin falla por una entrada no numérica, el menú puede entrar en un bucle; se podría validar con cin.fail() y cin.clear().
- Adaptación al P.I.A.: reemplazar Partida por un registro del prototipo (por ejemplo lecturas de sensores con fecha y hora).

11. Control de versiones

El repositorio se sincroniza con GitHub y GitLab (push dual). El historial de cambios está en CHANGELOG.md y el versionado sigue el formato v1.0.0.

```
git add .
git commit -m "Actividad 7 v1.0.0"
git push origin main
git push gitlab main
```